

This section outlines the overall strategy, key technology choices, and fundamental design principles that will guide the rewrite of the Teaching Vacancies platform.

4.1. Overall Approach

The project undertakes a complete rewrite of the existing Teaching Vacancies platform from its current Ruby on Rails technology stack to a new stack based on Python and Django. The primary goal is to achieve functional parity with the existing system while leveraging the benefits of the new technology stack, such as improved maintainability, access to a wider pool of developer talent, and alignment with current DfE technology strategies. The rewrite will involve migrating the existing data to a new database schema compatible with Django and rebuilding all user-facing features and backend processes.

4.2. Key Technology Choices

The selection of technologies aims to build a robust, scalable, and maintainable platform, aligning with modern best practices and available DfE-supported tools:

- **Programming Language:** Python (version 3.10+).
- **Web Framework:** Django (latest stable version, e.g., 5.x), providing a mature and comprehensive framework for web application development.
- **API Development:** Django REST framework (DRF) for building any necessary web APIs (e.g., for external ATS integration or future frontend requirements).
- **Database:** PostgreSQL (latest stable version) with the PostGIS extension to handle geospatial data for location-based searches and organisation mapping.
- **Asynchronous Task Processing:** Celery with Redis as the message broker and results backend, for handling background tasks such as sending email notifications, data synchronization, and analytics processing.
- **Authentication:**
 - `django-allauth` will be utilized to manage local account creation and authentication flows, particularly for jobseekers.
 - Integration with DfE Sign-In (DSI) via OAuth/OpenID Connect for publisher and support user authentication.
 - Integration with GOV.UK One Login via OpenID Connect for jobseeker authentication.

- **Email Notifications:** GOV.UK Notify will be the primary service for sending all transactional emails to users.
- **Frontend Bundling:** esbuild (or a similar modern JavaScript bundler like Webpack/Rollup if specific needs arise) will be used for compiling and bundling frontend assets (JavaScript, SCSS/CSS).
- **File Storage:** Cloud-based object storage (e.g., AWS S3 or equivalent, managed via `django-storages`) for user-uploaded files like organisation logos and job application documents.
- **Search:** Initially leveraging PostgreSQL's built-in full-text search capabilities (`SearchVectorField`). Further enhancements with dedicated search engines (e.g., OpenSearch) may be considered post-MVP if performance requirements dictate.
- **Geospatial Data Handling:** GeoDjango (part of Django) will be used for managing and querying geospatial data (e.g., organisation locations, vacancy search radii).

4.3. Fundamental Design Principles

The development will be guided by the following principles to ensure a high-quality, sustainable, and effective system:

- **Modular Application Structure:**
 - The system will be organized into distinct Django apps, each responsible for a specific domain or functional area (e.g., `users`, `organisations`, `vacancies`, `job_applications`, `jobseekers`, `publishers`, `notifications`, `api`). This promotes separation of concerns, improves maintainability, and allows for clearer ownership of code.
- **Service Layer:**
 - Complex business logic and operations that go beyond simple CRUD actions on models will be encapsulated within service classes or modules. This keeps models relatively lean (focused on data representation and basic operations) and controllers/views focused on request handling and presentation, improving testability and reusability of business logic.
- **Adherence to GOV.UK Design System:**
 - The user interface and user experience (UI/UX) will strictly adhere to the GOV.UK Design System guidelines and utilize its frontend components. This ensures consistency with other government digital services, promotes accessibility, and provides a familiar experience for users.
- **Test-Driven Development (TDD) / Comprehensive Testing:**
 - A strong emphasis will be placed on automated testing. This includes:

- **Unit Tests:** For individual functions, methods, and classes (including models, forms, services).
- **Integration Tests:** To verify interactions between different components (e.g., views, services, and database).
- **End-to-End (E2E) Tests:** For critical user journeys, simulating real user interactions with the browser.
- TDD or BDD practices are encouraged to ensure that code is written with testability in mind from the outset. High test coverage targets will be set and monitored.
- **Security by Design:**
 - Security considerations will be integrated throughout the development life-cycle, not as an afterthought. This includes:
 - Following OWASP Top 10 recommendations.
 - Implementing appropriate input validation, output encoding, and protection against common web vulnerabilities (XSS, CSRF, SQL injection, etc.).
 - Secure handling of authentication, authorization, and session management.
 - Regular security audits and penetration testing.
 - Encryption of sensitive data at rest and in transit.
- **Clean Code and Maintainability:**
 - Code will adhere to Python's PEP 8 style guide.
 - Tools like Black (formatter), Flake8/Ruff (linter), and Isort (import sorter) will be used to enforce consistency and quality.
 - Type hinting (as supported by MyPy) will be employed to improve code clarity, catch errors early, and aid refactoring.
 - Code will be well-documented, especially for complex logic, public APIs, and architectural components.
- **Iterative Development (within the rewrite context):**
 - While the overall strategic goal is a full rewrite achieving feature parity, the implementation will be broken down into manageable, iterative chunks. This could be based on user roles (e.g., jobseeker flows, then publisher flows), feature sets (e.g., search and alerts, then application management), or data entities (e.g., organisation and vacancy management first). This allows for incremental progress, easier testing of specific parts, and potentially earlier feedback loops if phased rollouts are considered.
- **Accessibility First:**

- The platform will be designed and developed to meet WCAG 2.1 AA standards from the outset, ensuring it is usable by people with a wide range of disabilities.
- **Configuration over Code:**
- Where possible, system behaviors will be managed through configuration (e.g., environment variables, Django settings) rather than hardcoding values, to improve flexibility across different environments.
- **Pragmatism and Simplicity (YAGNI/KISS):**
- Solutions will be chosen based on their ability to meet requirements effectively and simply, avoiding over-engineering or unnecessary complexity ("You Ain't Gonna Need It", "Keep It Simple, Stupid").